KeenPay

Production Database Schema

Multi-tenant, RLS-protected PostgreSQL for agentic commerce.
AI proposes. The control plane authorizes. Money never bypasses policy.

Grow | Sell | Protect | Protocol Gateway

PostgreSQL 15 | Integer Paise | Append-Only Audit | Row-Level Security

Version 2.0.0 | August 2026

1. Database Overview

KeenPay uses PostgreSQL as the single source of truth for tenants, catalog, negotiation sessions, orders, authorization, and audit. The schema is multi-tenant: every tenant-scoped row carries merchant_id (the tenant boundary) and Row-Level Security (RLS) isolates access. The AI runtime reads catalog and writes session proposals; financial tables are owned by the deterministic control plane.

Table Groups

Group	Tables
Tenancy & Identity	tenants, users, agents, merchant_policies
Catalog	products
Agentic Checkout	negotiation_sessions, langgraph_checkpoints
Commerce	orders, inventory_holds, refunds
Authorization & Safety	authorizations, escalation_tickets
Payments & Reliability	webhook_events, idempotency_keys, outbox_events
Audit	audit_logs (append-only)

Design Principles

- Integer paise (INTEGER) for all money - no floats
- TENANT = merchant_id; RLS isolates every tenant-scoped row
- AI may propose offers in negotiation_sessions; only policy-approved amounts reach orders
- audit_logs is append-only; UPDATE/DELETE blocked by database trigger
- Every payment link binds to guardrail_decision_id + offer_version + authorization_id
- Webhook events deduplicated by unique event_id; outbox guarantees side-effect delivery

2. Data Plane Architecture

The schema enforces a strict split. Protocol traffic enters through an Agent identity; LangGraph mirrors conversational state in negotiation_sessions and langgraph_checkpoints. Once guardrails approve and the user confirms, an immutable order row is created, optionally bound to a scoped authorization. Payment lifecycle completes via Razorpay webhooks stored in webhook_events.

UNTRUSTED INPUT	User chat, LLM proposals, inbound webhooks
PROTOCOL GATEWAY	agents, scopes, authorizations (identity + mandates)
LANGGRAPH STATE	negotiation_sessions, langgraph_checkpoints
CONTROL PLANE	Policy binding via guardrail_decision_id + authorization_id
DURABLE AUDIT	audit_logs, webhook_events, escalation_tickets, outbox_events

Entity Relationships (v2)

tenants <- agents / policies / products <- negotiation_sessions (via selected_line_items JSONB) -> orders -> webhook_events / refunds. authorizations bind a scoped mandate to a cart_hash and an amount window, then link to orders.authorization_id. audit_logs correlates session_id, order_id, decision_id, and protocol_ref across the lifecycle. inventory_holds tie stock reservations to a session before payment link creation. outbox_events guarantee async side-effects.

3. Tenancy & Identity

TABLE: tenants

Purpose

Tenant registry. All tenant-scoped tables reference this id as merchant_id - the tenant boundary and RLS key. The registry itself is read-only to applications.

Column	Type	Meaning
id	VARCHAR(64)	Tenant id (merchant_id referenced everywhere)
name	VARCHAR(255)	Display / legal name
status	ENUM	active | suspended
currency	CHAR(3)	ISO-4217; INR in v1
locale	VARCHAR(16)	Default locale, e.g. en-IN
metadata	JSONB	Billing, contact, feature flags

Foreign Keys

- none (root of the tenant tree)

Important Indexes

- (status)
- (created_at DESC)

Service Access

- AI Runtime: yes
- Policy Engine: yes
- Payment Service: no direct access
- Admin API: yes

TABLE: users

Purpose

Shoppers & their identity. PII is never placed in trace payloads; only an anonymized id is shared with the LLM. Email/phone uniqueness is scoped per tenant.

Column	Type	Meaning
id	VARCHAR(64)	User id (referenced by negotiation_sessions.user_id / orders.user_id)
merchant_id	VARCHAR(64)	Tenant scope
email	VARCHAR(255)	Latest verified email (nullable)
phone	VARCHAR(32)	Latest verified phone (nullable)
external_id	VARCHAR(128)	Identity-provider reference
status	ENUM	active | suspended | deleted
metadata	JSONB	Preferences, LTV hints, tags

Foreign Keys

- merchant_id -> tenants.id

Important Indexes

- (merchant_id, created_at DESC)
- UNIQUE (merchant_id, email) WHERE email IS NOT NULL
- UNIQUE (merchant_id, phone) WHERE phone IS NOT NULL
- (external_id)

Service Access

- AI Runtime: yes
- Policy Engine: yes
- Payment Service: no direct access
- Merchant Dashboard: yes

TABLE: agents

Purpose

Protocol Gateway identity. MCP/A2A tools must appear in allowed_tools; trust_level 1 is propose-only, 5 is trusted automation. No agent can hold payment credentials.

Column	Type	Meaning
id	VARCHAR(64)	Agent id (referenced by negotiation_sessions.agent_id)
merchant_id	VARCHAR(64)	Tenant scope
agent_type	ENUM	agent | protocol | integration | system
name	VARCHAR(255)	Display name
protocol	VARCHAR(32)	MCP | ACP | UCP | AP2 | A2A | x402 | A2UI | UPI | NULL
scopes	JSONB	Granted scopes, e.g. ["checkout.write"]
allowed_tools	JSONB	Tool allowlist - never payment tools
trust_level	INTEGER	1 (propose) .. 5 (trusted)
status	ENUM	active | disabled | expired
expires_at	TIMESTAMPTZ	Optional credential expiry

Foreign Keys

- merchant_id -> tenants.id

Important Indexes

- (merchant_id, status)
- (protocol)
- (merchant_id, agent_type)

Service Access

- Protocol Gateway: yes
- AI Runtime: yes
- Policy Engine: no direct access
- Payment Service: no direct access

TABLE: merchant_policies

Purpose

Persisted MerchantPolicy config (MERCHANT_POLICY_JSON). Exactly one active row per tenant. Changes are audited in audit_logs.

Column	Type	Meaning
id	UUID	Policy row id
merchant_id	VARCHAR(64)	Tenant scope
policy_version	VARCHAR(32)	e.g. 2026.08.1
is_active	BOOLEAN	Only one active per tenant (partial unique index)
config	JSONB	Discount caps, margin floor, qty limits, rounds, thresholds

Foreign Keys

- merchant_id -> tenants.id

Important Indexes

- UNIQUE (merchant_id, policy_version)
- UNIQUE (merchant_id) WHERE is_active

Service Access

- AI Runtime: no direct access
- Policy Engine: yes
- Merchant Dashboard: yes

4. Catalog Tables

TABLE: products

Purpose

Merchant catalog. Price and cost basis are read by the policy engine for margin guardrails. The AI runtime may read active products but cannot mutate prices.

Column	Type	Meaning
id	VARCHAR(64)	Product ID
sku	VARCHAR(64)	Stock keeping unit (unique per tenant)
merchant_id	VARCHAR(64)	Tenant scope (default merchant_keen)
name	VARCHAR(255)	Display name
list_price_paise	INTEGER	List price in paise
cost_paise	INTEGER	Cost basis for RULE_MIN_MARGIN
wholesale_paise	INTEGER	Optional wholesale cost basis
quantity_on_hand	INTEGER	Physical stock
quantity_reserved	INTEGER	Reserved by active holds
attributes	JSONB	Category, color, size, etc.
search_vector	TSVECTOR	Full-text search index (generated)
active	BOOLEAN	Catalog visibility

Foreign Keys

- merchant_id -> tenants.id

Important Indexes

- (merchant_id, sku) UNIQUE
- GIN on search_vector
- GIN on attributes
- (merchant_id, active)

Service Access

- AI Runtime (LangGraph): yes
- Policy Engine: yes
- Payment Service: no direct access
- Webhook Worker: no direct access

5. Agentic Checkout Tables

TABLE: negotiation_sessions

Purpose

Live checkout session. Mirrors LangGraph KeenPayState: intent, offers, guardrail binding, user confirmation gate, and protocol provenance. No payment link without APPROVED guardrail + user_confirmed_payment.

Column	Type	Meaning
id	UUID	Session ID (= LangGraph thread_id)
merchant_id	VARCHAR(64)	Tenant scope
user_id	VARCHAR(64)	Optional shopper identity
status	ENUM	active | negotiating | awaiting_confirmation | payment_pending | paid | escalated | closed
negotiation_round	INTEGER	Bounded negotiation counter (max 5)
offer_version	INTEGER	Monotonic offer version
parsed_intent	JSONB	Structured intent from parse_intent node
proposed_offer	JSONB	LLM-proposed offer (untrusted until guardrail)
approved_offer	JSONB	Policy-approved offer snapshot
guardrail_decision	ENUM	APPROVED | REJECTED | ESCALATED
guardrail_decision_id	UUID	Links to audit_logs decision
guardrail_detail	JSONB	Per-rule evaluation results
user_confirmed_payment	BOOLEAN	Explicit user consent gate
final_amount_paise	INTEGER	Deterministic total (post compute_totals)
security_block	BOOLEAN	True after prompt injection / anomaly block

agent_id	VARCHAR(64)	Originating protocol agent (null = native web)
protocol_ref	VARCHAR(128)	External protocol reference id
langgraph_thread_id	UUID	LangGraph checkpointer reference

Foreign Keys

- merchant_id -> tenants.id
- user_id -> users.id
- agent_id -> agents.id

Important Indexes

- (user_id, created_at DESC)
- (status) partial
- (merchant_id, created_at DESC)
- (agent_id) WHERE agent_id IS NOT NULL
- (guardrail_decision_id)

Service Access

- AI Runtime (LangGraph): yes
- Policy Engine: yes
- Payment Service: no direct access
- Merchant Dashboard: yes

TABLE: langgraph_checkpoints

Purpose

LangGraph durable state snapshots for conversation recovery and interrupt/resume.

Column	Type	Meaning
thread_id	UUID	Session thread
merchant_id	VARCHAR(64)	Tenant scope
checkpoint_id	UUID	Checkpoint version
checkpoint	JSONB	Serialized graph state
metadata	JSONB	Node hints, timestamps

Foreign Keys

- thread_id -> negotiation_sessions.langgraph_thread_id
- merchant_id -> tenants.id

Important Indexes

- (thread_id, created_at DESC)
- (merchant_id, created_at DESC)

Service Access

- AI Runtime (LangGraph): yes
- Policy Engine: no direct access
- Payment Service: no direct access

6. Commerce & Inventory Tables

TABLE: orders

Purpose

Frozen purchase created only after guardrail APPROVED + user confirmation. Amounts and line_items are immutable after insert. Binds to Razorpay payment link and, optionally, a scoped authorization.

Column	Type	Meaning
id	VARCHAR(64)	Order ID
session_id	UUID	Source negotiation session
merchant_id	VARCHAR(64)	Tenant scope
user_id	VARCHAR(64)	Shopper identity
status	ENUM	pending | paid | expired | cancelled | payment_disputed | refunded
subtotal_paise	INTEGER	Pre-discount subtotal
discount_amount_paise	INTEGER	Applied discount
final_amount_paise	INTEGER	Charge amount (must match webhook)
line_items	JSONB	Immutable line snapshot at order time
guardrail_decision_id	UUID	Required policy evaluation reference
offer_version	INTEGER	Offer version that created this order
policy_version	VARCHAR(32)	Policy ruleset version
authorization_id	UUID	Optional scoped authorization that authorized the charge
razorpay_payment_link_id	VARCHAR(64)	Razorpay entity
razorpay_payment_link_url	TEXT	Checkout URL for buyer
idempotency_key	VARCHAR(128)	Duplicate protection (UNIQUE)

Foreign Keys

- session_id -> negotiation_sessions.id
- merchant_id -> tenants.id
- user_id -> users.id
- authorization_id -> authorizations.id (logical)

Important Indexes

- (session_id)
- (status, created_at DESC)
- (razorpay_payment_link_id) partial
- (authorization_id) WHERE authorization_id IS NOT NULL
- UNIQUE (idempotency_key)
- UNIQUE pending order per session

Service Access

- AI Runtime: no direct access
- Payment Service: yes
- Webhook Worker: yes
- Reconciliation Worker: yes

TABLE: inventory_holds

Purpose

Durable stock reservation during checkout. Complements Redis soft holds. Held rows carry the tenant scope so a worker can release across tenants safely.

Column	Type	Meaning
id	UUID	Hold ID
session_id	UUID	Checkout session
product_id	VARCHAR(64)	Product reserved
merchant_id	VARCHAR(64)	Tenant scope
sku	VARCHAR(64)	SKU for guardrail reads
quantity	INTEGER	Reserved quantity
expires_at	TIMESTAMPTZ	Auto-release time (15 min default)
released_at	TIMESTAMPTZ	Null while active

Foreign Keys

- session_id -> negotiation_sessions.id
- product_id -> products.id
- merchant_id -> tenants.id

Important Indexes

- (expires_at) WHERE released_at IS NULL
- (product_id) WHERE released_at IS NULL
- (merchant_id) WHERE released_at IS NULL
- UNIQUE (session_id, sku)

Service Access

- AI Runtime: no direct access
- Policy Engine: yes
- Payment Service: yes

TABLE: refunds

Purpose

Money-back record. Amount is validated against the order before a refund is initiated; the status tracks the provider lifecycle.

Column	Type	Meaning
id	UUID	Refund ID
order_id	VARCHAR(64)	Source order
merchant_id	VARCHAR(64)	Tenant scope
amount_paise	INTEGER	Refund amount in paise (> 0)
currency	CHAR(3)	ISO currency (INR v1)
razorpay_refund_id	VARCHAR(64)	Provider refund reference
reason	VARCHAR(255)	Human-readable reason
status	ENUM	created | initiated | completed | failed
initiated_by	VARCHAR(64)	agent / user / human operator
notes	JSONB	Audit-relevant context

Foreign Keys

- order_id -> orders.id
- merchant_id -> tenants.id

Important Indexes

- (order_id, created_at DESC)
- (merchant_id, created_at DESC)

Service Access

- AI Runtime: no direct access
- Payment Service: yes
- Merchant Dashboard: yes

7. Authorization & Safety

TABLE: authorizations

Purpose

Scoped, expiring, single-use payment mandate (e.g. AP2 verifiable authorization). Bound to a cart_hash + amount window so the amount cannot be changed between issue and settle.

Column	Type	Meaning
id	UUID	Authorization ID
merchant_id	VARCHAR(64)	Tenant scope
agent_id	VARCHAR(64)	Issuing protocol agent
session_id	UUID	Checkout session (nullable)
authorization_type	VARCHAR(32)	e.g. AP2_MANDATE, PAYMENT
cart_hash	VARCHAR(64)	Hash of line_items (tamper-proof binding)
amount_paise_min	INTEGER	Lower bound on authorize amount
amount_paise_max	INTEGER	Upper bound on authorize amount
scope	JSONB	Granted permissions / ordering terms
protocol_ref	VARCHAR(128)	External mandate reference
status	ENUM	issued | consumed | expired | revoked | denied
expires_at	TIMESTAMPTZ	Mandate expiry
consumed_at	TIMESTAMPTZ	Set when consumed by a charge

Foreign Keys

- merchant_id -> tenants.id
- agent_id -> agents.id
- session_id -> negotiation_sessions.id

Important Indexes

- (merchant_id, status, expires_at)
- (cart_hash)
- (session_id)
- (agent_id)

Service Access

- Protocol Gateway: yes
- Policy Engine: yes
- Payment Service: yes
- AI Runtime: no direct access

TABLE: escalation_tickets

Purpose

Human-in-the-loop queue when guardrails ESCALATE (anomaly, max rounds, engine error). Human overrides are logged with actor=human; margin floor is never overridden in v1.

Column	Type	Meaning
id	UUID	Ticket ID
session_id	UUID	Blocked session
merchant_id	VARCHAR(64)	Tenant scope
priority	VARCHAR(4)	P0 | P1 | P2
reason_code	VARCHAR(64)	e.g. RULE_SECURITY_ANOMALY
status	VARCHAR(16)	open | assigned | resolved | expired
assigned_to	VARCHAR(64)	Handler identity
proposed_offer_snapshot	JSONB	Offer under review
policy_snapshot	JSONB	Policy at escalation time
resolution	VARCHAR(32)	approve_override | deny | counter_offer
override_discount_pct	NUMERIC(5,2)	Manager override (bounded)

Foreign Keys

- session_id -> negotiation_sessions.id

- merchant_id -> tenants.id

Important Indexes

- (status, priority, created_at)
- (merchant_id, status, created_at)

Service Access

- AI Runtime: no direct access
- Control Plane / Admin API: yes
- Merchant Dashboard: yes

8. Payments & Reliability

TABLE: webhook_events

Purpose

Razorpay inbound events. Signature verified on receipt; deduplicated by event_id. Amount mismatch marks order payment_disputed - never auto-paid.

Column	Type	Meaning
id	UUID	Internal event ID
event_id	VARCHAR(128)	Razorpay event ID (UNIQUE)
merchant_id	VARCHAR(64)	Tenant scope
event_type	VARCHAR(64)	e.g. payment_link.paid
payload	JSONB	Raw webhook body
signature_valid	BOOLEAN	HMAC verification result
processed	BOOLEAN	Worker completion flag
order_id	VARCHAR(64)	Matched order

Foreign Keys

- order_id -> orders.id
- merchant_id -> tenants.id

Important Indexes

- UNIQUE (event_id)
- (event_type, received_at DESC)
- unprocessed index
- (merchant_id, received_at DESC)

Service Access

- AI Runtime: no direct access
- Webhook Worker: yes
- Payment Service: no direct access

TABLE: idempotency_keys

Purpose

Server-side idempotency cache. The same idempotency_key + scope returns the same response; prevents duplicate charges and double payment links across retries.

Column	Type	Meaning
id	UUID	Key row ID
merchant_id	VARCHAR(64)	Tenant scope
scope	VARCHAR(64)	Operation class, e.g. payment_link, webhook
idempotency_key	VARCHAR(128)	Client/protocol-provided key
request_hash	VARCHAR(64)	Hash of request body (mismatch -> error)
response_status	INTEGER	Cached HTTP status
response_body	JSONB	Cached response
expires_at	TIMESTAMPTZ	Cache TTL

Foreign Keys

- merchant_id -> tenants.id

Important Indexes

- UNIQUE (merchant_id, scope, idempotency_key)
- (expires_at)

Service Access

- API Gateway: yes
- Payment Service: yes
- Webhook Worker: yes
- AI Runtime: no direct access

TABLE: outbox_events

Purpose

Transactional outbox. The business transaction writes the MBT row + an outbox row atomically; a worker publishes to the bus. Guarantees no lost side-effects (webhooks, notifications, SQS).

Column	Type	Meaning
id	UUID	Event row ID
merchant_id	VARCHAR(64)	Tenant scope
event_type	VARCHAR(128)	e.g. order.paid, link.created
aggregate_type	VARCHAR(64)	order, refund, authorization
aggregate_id	VARCHAR(128)	Aggregate reference
payload	JSONB	Event body
status	ENUM	pending | published | failed | dead
attempts	INTEGER	Delivery attempt count
published_at	TIMESTAMPTZ	Set when published

Foreign Keys

- merchant_id -> tenants.id

Important Indexes

- (status, created_at) WHERE status = 'pending'
- (aggregate_type, aggregate_id, created_at)

Service Access

- AI Runtime: no direct access
- Outbox Worker: yes
- Reconciliation Worker: yes

9. Audit

TABLE: audit_logs

Purpose

Append-only tamper-evident ledger. Every money action, guardrail evaluation, and protocol decision writes a row with input_snapshot and output_snapshot. UPDATE/DELETE blocked by trigger.

Column	Type	Meaning
id	UUID	Audit row ID
session_id	UUID	Negotiation session
order_id	VARCHAR(64)	Order if applicable
merchant_id	VARCHAR(64)	Tenant scope
actor	ENUM	agent | protocol | policy_engine | user | system | webhook | human
action	VARCHAR(128)	e.g. GUARDRAIL_EVALUATED, PAYMENT_LINK_CREATED
decision_id	UUID	Guardrail evaluation reference
offer_version	INTEGER	Bound offer version
protocol_ref	VARCHAR(128)	Originating protocol reference
input_snapshot	JSONB	Proposed offer, policy version, message hash
output_snapshot	JSONB	Decision, rule results, payment link ID
trace_metadata	JSONB	Node name, duration_ms, anomaly score

Foreign Keys

- session_id -> negotiation_sessions.id
- order_id -> orders.id
- merchant_id -> tenants.id

Important Indexes

- (session_id, created_at DESC)
- (order_id) partial
- (decision_id) partial
- GIN on trace_metadata
- (actor, action, created_at DESC)

Service Access

- AI Runtime: no direct access
- Policy Engine: yes
- Merchant Dashboard (read): yes
- Webhook Worker: yes

10. Row-Level Security (Tenant Isolation)

Every tenant-scoped table enables RLS. An application role (keenpay_app) must set the tenant context before DML using SET LOCAL app.tenant_id = '<tenant_id>'. Superusers and table owners bypass RLS, so local development is unaffected; production uses a non-owner role so policies are enforced. If no tenant context is set, policies fail closed (return no rows).

Tenancy Policy

Policy	Guard
rls_check(tenant_key)	tenant_key = current_tenant_id() OR app.bypass_rls = 'true'
current_tenant_id()	nullif(current_setting('app.tenant_id', true), '')
USING (SELECT/UPDATE/DELETE)	Tenant scope only
WITH CHECK (INSERT)	Tenant scope only (prevents cross-tenant writes)

Protected Tables

`users`	Tenant identity
`agents`	Protocol Gateway actors
`merchant_policies`	Per-tenant policy config
`products`	Catalog + margin basis
`negotiation_sessions`	Checkout session mirror
`orders`	Money: immutable after insert
`authorizations`	Scoped mandates
`idempotency_keys`	Retry / dedupe
`audit_logs`	Append-only ledger
`inventory_holds`	Stock reservations
`webhook_events`	Provider webhooks
`escalation_tickets`	HITL queue
`langgraph_checkpoints`	Graph snapshots
`refunds`	Money back
`outbox_events`	Guaranteed side-effects

Production roles

keenpay_app - non-owner runtime role; RLS is enforced for every query.
keenpay_admin - admin / migration role; may bypass RLS for maintenance.
keeps secrets in environment or AWS Secrets Manager - never in the DB or LLM context.

11. State Transitions

negotiation_sessions.status

From	To / Action
active	negotiating
negotiating	awaiting_confirmation (guardrail APPROVED)
awaiting_confirmation	payment_pending (user confirmed)
payment_pending	paid (webhook verified)
*	escalated (guardrail ESCALATED)
*	closed (terminal)

orders.status

From	To / Action
pending	paid (webhook amount match)
pending	expired (link TTL)
pending	cancelled
pending	payment_disputed (amount mismatch)
paid	refunded (refund completed)

authorizations.status

From	To / Action
issued	consumed (charge bound to mandate)
issued	expired (TTL reached)
issued	revoked (policy / user)
issued	denied (policy / risk)

guardrail_decision

From	To / Action
proposed offer	APPROVED -> compute_totals
proposed offer	REJECTED -> explain + retry (max 5 rounds)
proposed offer	ESCALATED -> escalation_tickets

SELL Database Flow

- 1. Protocol/chat -> negotiation_sessions updated (proposed_offer, agent_id, protocol_ref)
- 2. Policy engine -> guardrail_decision + authorization_id + audit_logs row
- 3. User confirms -> inventory_holds + orders (pending) + idempotency_key + outbox row
- 4. Webhook -> webhook_events + orders.status=paid + audit_logs + outbox publish
- 5. Refund / dispute -> refunds + orders.refunded / payment_disputed + audit_logs

12. Transaction Passport (Derived View)

There is no separate passport table. The Transaction Passport is assembled from negotiation_sessions, orders, authorizations, audit_logs, and webhook_events for support replay and compliance. The frontend trace panel streams real-time events via Redis; audit_logs is the durable source of truth.

Source of Truth

Thing	Where it lives
Product price	products.list_price_paise
Negotiated unit price	approved_offer in negotiation_sessions
Final charge amount	orders.final_amount_paise
Policy evaluation	audit_logs where action=GUARDRAIL_EVALUATED
Scoped mandate	authorizations (cart_hash + amount window)
Payment status	orders.status + webhook_events
Provider truth	Razorpay API + verified webhook
Protocol provenance	negotiation_sessions.agent_id + protocol_ref
Refund state	refunds.status
Hot session cache	Redis (never authoritative for money)

13. Production Checklist

- All money stored as integer paise - no floating point
- merchant_id (tenant) on every tenant-scoped table; RLS enabled + policies present
- Payment link requires guardrail_decision=APPROVED + user_confirmed_payment
- Scoped authorization consumed before charge; amount within min/max window
- Idempotency key on every order; durable idempotency_keys cache per scope
- Webhook HMAC verified; duplicate event_id returns 200 without side effect
- Webhook amount = orders.final_amount_paise or mark payment_disputed (no auto-complete)
- audit_logs append-only (trigger blocks UPDATE/DELETE)
- Inventory: quantity_reserved <= quantity_on_hand constraint
- LLM never writes to orders / authorizations / webhook_events - gated Python nodes only
- Outbox write is transactional with the business mutation to avoid lost side-effects
- Negotiation capped at 5 rounds before ESCALATED -> escalation_tickets
- Secrets in environment / secrets manager - never in database or LLM context
- Refund amount validated against the order before initiate

14. Production Hardening (Included)

This DDL ships multi-tenant isolation, agent/policy/authorization tables, durable idempotency, a transactional outbox, and refunds - the items previously deferred to v1.1. v1 policy still encodes discount/margin caps in merchant_policies.config and snapshots guardrail evaluations in audit_logs JSONB. GROW campaign tables remain out of scope for the storefront pilot.